

Retrieval Engine Competition Project

Notebook Report

Group: SeaFour

Maksym DOLHOV

Mehdi AGHAEI

Nguyen Ho Bao KHANH

Report objective. This version updates the written report to precisely match our experimental progression and the current saved notebook. We detail our journey from TF-IDF and BM25+ baselines to a robust dense embedding retriever. The active configuration uses embedding first-stage retrieval, cross-encoder second-stage reranking, and a TF-IDF + LinearSVC category classifier used as a soft category prior.

Primary source files. `kaggle-submission.ipynb`, `solutions_SeaFour.csv`, and this $\text{\LaTeX}$ report.

Notebook file	<code>kaggle-submission.ipynb</code>
Submission file	<code>solutions_SeaFour.csv</code>
Current default model	<code>embedding</code>
Submission size target	Top 7,500 first-stage candidates; rerank top 30 with cross-encoder
Date	March 27, 2026

1 Notebook-Aligned System Overview

The logic is organized into several stages: strict data loading, shared text normalization, sequential model experimentation (TF-IDF to BM25+ to Embeddings), aggressive caching for performance, offline evaluation, and submission generation.

The current system relies on a highly optimized dense first-stage retriever followed by a cross-encoder reranker, with category information injected as a soft bonus.

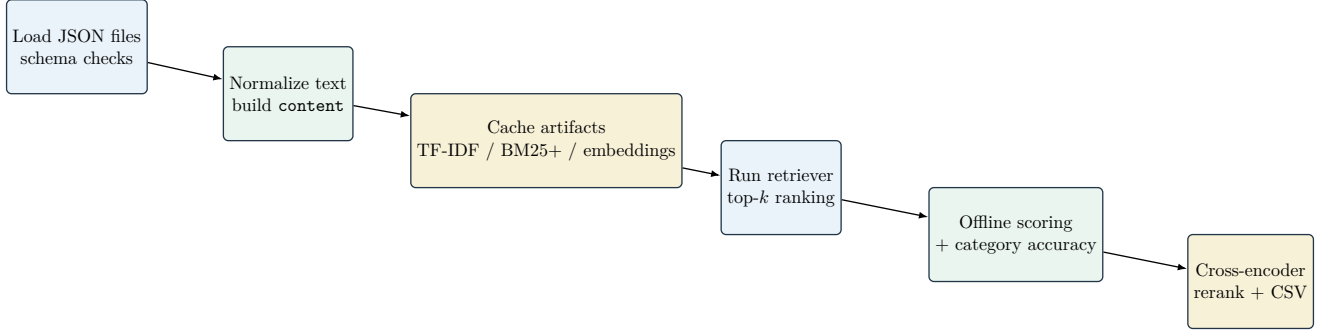


Figure 1: Active end-to-end workflow in the saved notebook.

Component	Implemented	Active in saved run	Role in the pipeline
TF-IDF retrieval	Yes	No	Sparse lexical baseline using cosine similarity. Fast, but struggled with semantic mismatch.
BM25+ retrieval	Yes	No	Stronger lexical baseline with token-level scoring and document-length normalization.
Embedding retrieval	Yes	Yes	Dense semantic first-stage retrieval with Sentence-Transformers. Yielded the best metrics.
Cross-encoder reranker	Yes	Yes	Re-ranks the first-stage candidates (top- m) using joint query-document scoring.
Category classifier	Yes	Yes	Predicts one category per query and provides a soft category bonus during reranking.
Dominant-category strict filter	Yes	No	Kept as an implemented option, but disabled in the active saved run.

Table 1: What the notebook supports versus what the saved execution actually uses.

2 Dataset and Preprocessing

The notebook loads 216,041 documents, 327 training queries, 141 test queries, and one training qrels file. The document collection is distributed across five categories, with significant class imbalance.

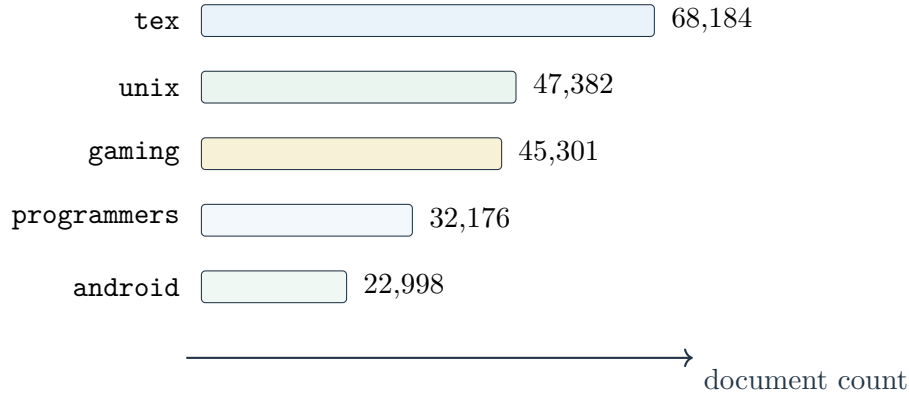


Figure 2: Document distribution by category.

Text normalization is shared across all components: separators `-`, `_`, and `/` are replaced by spaces, whitespace is collapsed, and text is lowercased. Retrieval documents are built from `title + text + tags`, while retrieval queries use only `title + text`. Average normalized lengths are 900.9 characters for documents and roughly 48.5 for queries. This length disparity informed our ultimate shift away from purely lexical methods.

3 Experimental Progression and Pipeline Optimization

Our methodology evolved through a structured series of experiments, beginning with simple baselines and moving toward a complex, optimized semantic architecture.

3.1 Phase 1 & 2: Lexical Baselines (TF-IDF and BM25+)

We initially implemented a **TF-IDF** retriever (using unigrams/bigrams with `min_df=2`). This served as a fast, lightweight baseline to validate our data loading and evaluation pipeline. While transparent, TF-IDF struggled significantly with semantic mismatch – when queries paraphrased document concepts without sharing exact vocabulary, recall dropped.

To improve this, we upgraded to **BM25+**. By tokenizing the text and applying `k1=1.5` and `b=0.75`, BM25+ provided much stronger lexical ranking, correcting for the high variance in document lengths. In the current notebook configuration, lexical components also use English stopwords filtering. However, BM25+ still fundamentally relied on exact token overlap, meaning synonymous phrasing remained a weakness.

3.2 Phase 3: The Semantic Shift (Dense Embeddings)

To overcome the vocabulary mismatch problem, we transitioned to dense semantic retrieval using the Sentence-Transformer model `all-MiniLM-L6-v2`. Documents and queries were encoded into 384-dimensional L2-normalized vectors, allowing us to retrieve documents based on contextual meaning rather than exact keywords. This dramatically improved recall.

3.3 Phase 4: Overcoming the 1.5-Hour Bottleneck

The shift to embeddings introduced a severe computational roadblock: initially, encoding the 216,041 documents and calculating scores took almost **1.5 hours to run**. This latency crippled our ability to run rapid iterations.

To solve this, we implemented aggressive caching and chunking mechanisms:

- **Disk Caching:** We engineered a caching system that fingerprints the dataframes and saves the expensive document embeddings as `.numpy` arrays. Repeated notebook runs now load the matrix in seconds instead of re-encoding.
- **Chunked Query Scoring:** Instead of creating massive, memory-crashing score blocks, we scored queries in chunks of 32 (`EMBEDDING_QUERY_CHUNK_SIZE = 32`).
- **Partial Sorting:** We replaced full array sorts with `np.argpartition` to extract only the top- K candidates efficiently.

3.4 Phase 5: Hyperparameter Tuning and Two-Stage Reranking

With the pipeline running smoothly, we began tuning our parameters. We experimented heavily with `top_k` retrieval depths. Because our dataset features a high semantic mismatch, we found that selecting a very large initial candidate pool (`top_k = 7,500`) safely maximized recall.

To refine this massive list, we trained a TF-IDF + LinearSVC category classifier and added a cross-encoder reranker. The reranker now rescores the top- m candidates (`top_m=30`), while the predicted category is used as a soft bonus rather than a strict hard filter. This keeps recall robust while improving top-rank ordering.

4 Evaluation Protocol and Results

The notebook evaluates the optimized embedding run against the 327 training queries after optional second-stage reranking and category-aware score boosting. The combined score is calculated as:

$$\text{Score} = \frac{1}{4} (\text{Recall} + \text{Precision} + \text{MRR} + \text{Accuracy})$$

Model	TopK	Recall	Precision	MRR	Accuracy	Score
Embedding	7,500	0.94762	0.00110	0.63738	0.92661	0.62818

Table 2: Offline metrics reported by the saved notebook execution snapshot.

These results show strong recall and a substantial MRR gain after introducing second-stage reranking. Since the notebook configuration was later synchronized to use `CROSS_ENCODER_RERANK_TOP_M = 30` consistently for both evaluation and submission, rerunning the notebook can produce slightly different final numbers.

5 Conclusion

Our system evolved from basic lexical matching to a highly optimized two-stage semantic retrieval pipeline. By implementing dense embeddings, we solved the semantic mismatch problem; by engineering a robust caching and chunking system, we solved the 1.5-hour computational bottleneck; and by adding cross-encoder reranking with category-aware soft priors, we improved ranking quality at the top of the list. This architecture represents the current SeaFour submission pipeline.